

Object-Oriented Programming Using

C++



Lesson N6

Object-Oriented Programming Using C++

Contents

Operator Overloading ->	3
Concept of Smart Pointer	7
Functions with an Arbitrary Number and Type of Arguments.	11
Homework.	18

Operator Overloading ->

We hope you remember that in C++ you can reload all operators, except for a few of them. In any case, the operator -> is overloaded, and this is extremely important. By the way, this operator is called a member selector. Consider the following example:

```
#include <iostream>
using namespace std;
//class, encapsulation pointer

class Temp
{
    int TEMP;
public:
    //constructor
    Temp() {TEMP=25;}
    //function display on the screen
    void TempFunction() {
        cout<<"TEMP = "<<TEMP<<"\n\n";
    }
    //function of value setting
    void TempSet(int T) {
        TEMP=T;
    }
};

//class, encapsulation pointer
class MyPtr
{
    //pointer to class Temp
    Temp*ptr;
```

```

public:
    //constructor
    MyPtr(Temp*p=NULL){
        ptr=p;
    }

    //type conversion operator
    //from encapsulated to encapsulates
    operator Temp*(){
        return ptr;
    };
    //member selector ->
    //allows to refer
    //directly to "hidden" pointer
    Temp* operator->(){
        return ptr;
    }
    //operator ++ for the forward
    // displacement of the pointer
    MyPtr operator++(){
        ptr++;
        return *this;
    }
};

void main ()
{
    //creating of a new object
    Temp*main_ptr = new Temp;
    //a simple appeal to the members of the object
    //through the "native" pointer
    main_ptr->TempFunction();

    //creating an object of class pointer
    MyPtr pTemp(main_ptr);
    //treatment through the class pointer
    pTemp->TempFunction();
}

```

```

//creating an array of objects
//of encapsulation class
Temp*arr_=new Temp[3];

//filling aforesaid array
//values from 0 to 2
for(int i=0;i<3;i++) arr_[i].TempSet(i);

//creating a class object pointer
//and write to it the array address
//(type conversion employs)
MyPtr arr_temp=arr_;
//shift one element forward
arr_temp++;
//result demonstration
arr_temp->TempFunction();

//delete objects
delete main_ptr;
delete[]arr_;
}

```

Result of the program work

TEMP = 25

TEMP = 25

TEMP = 1

So, let us discuss the result: we have a Temp class that can have instances, it has a certain TEMP member and the minimum set of functions to work with him.

In addition, we have created a class of object pointer MyPtr, which stores a conventional pointer, but we restrict an access to it and we overload operators for it:

1. Type cast operator from Temp to MyPtr
2. Selector ->

3. Operator ++ implements the shift of the pointer one step forward.

Consider the positive things — we got the class of objects pointers that can be boldly taken instead of real ones. This is convenient, because all functions for working with a pointer can be encapsulated in this class.

However, there is one more widespread use of this structure, which we will introduce in the next section of the lesson. Let's go!

Concept of Smart Pointer

So, we are going to talk about something that is implemented by the selector overload. So.

Smart pointer is a class that simulates the interface of the conventional pointer and adds to it some new functionality, such as bounds checking when accessing or cleanup.

In other words, smart pointers are objects that store a pointer to the real object and, as a rule, a counter of the number of calls to the object.

In the class there is also a destructor for this object that is not called from outside, but only inside the smart pointer. The principle is that when the counter of the calls to the object is equal to 0, the destructor will be called.

For example, there is an array of pointers to objects. You use a copy of the pointer to one object anywhere else. Then you clean the array, deleting objects, and "somewhere else" the pointer is still stored. Naturally, that the accessing it causes the program crash, because the object has been removed when cleaning the array.

If there is such smart pointer, this will not happen, because when you create a copy of the pointer an internal counter is incremented by 1. Now it will be equal to two — one for the array, and another one for the copy. Now, even if we remove the object from the array, the internal counter is reduced by 1 and becomes equal to 1, that is, a copy can continue to work with the object. Once it becomes unnecessary, copy frees the

memory from it and the counter becomes equal to 0. This is when the destructor for the original object is caused.

Well, everything is said. It's time to taste the dish called "smart pointer".

```
#include <iostream>
using namespace std;
class Temp
{
    int TEMP;
public:
    //constructor
    Temp(){TEMP=25;}
    //function display on the screen
    void TempFunction(){
        cout<<"TEMP = "<<TEMP<<"\n\n";
    }
    //function of assignment value
    void TempSet(int T){
        TEMP=T;
    }
};

//class that implements a smart pointer
class SmartPointer
{
    //encapsulated pointer
    Temp*ptr;
    //copies counter
    int count_copy;

public:
    //constructor
    SmartPointer (Temp*p=NULL){
        //write 0 when creating copies no
        count_copy=0;
```



```

        ptr=p;
    }
    //copy constructor
    SmartPointer (const SmartPointer&obj){
        //create copy - increase counter
        ptr=obj.ptr;
        count_copy++;
    }
    //overload operator equals
    SmartPointer operator=(const SmartPointer&obj){
        //create copy - increase counter
        ptr=obj.ptr;
        ptr=obj.ptr;
        count_copy++;
        //return the object for the situation a=b=c
        return *this;
    }
    //object destruction
    ~SmartPointer(){
        //if there is an object and there are not copies
        if(ptr!=NULL&&count_copy==0){
            cout<<"\n~Delete Object\n";
            //destroy the object
            delete[]ptr;
        }
        //otherwise(copy is destroyed)
        else{
            //decrement counter
            count_copy--;
            cout<<"\n~Delete Copy\n";
        }
    }
    //overload selector
    Temp* operator->(){
        return ptr;
    }
};

void main(){
    //create an object
    Temp*main_ptr=new Temp;

```

```
//initialize smart pointer by this object
SmartPointer PTR(main_ptr);
//check the work of the smart pointer
PTR->TempSet(100);
PTR->TempFunction();
//create a copy (work of the copy constructor)
SmartPointer PTR2=PTR;
}
```

Result of the program work

```
//work with an object through a smart pointer
TEMP = 100
//destruction of the copy
~Delete Copy
//destruction of the object
~Delete Object
```

Now we have created our own pointer, in the following lessons we will consider a ready-made smart pointer from the STL that is called `auto_ptr`.

Functions with an Arbitrary Number and Type of Arguments

This section of the lesson is devoted to the study of a relatively new type of functions, the number of parameters in which becomes known only at the time of treatment to this function.

Thus, in C++ language are permissible functions for which the number and types of parameters at compile the function are not defined. These values become known only at the time of the function call that explicitly sets the list of actual parameters. When identifying and documenting such functions with lists of parameters of unknown length, the specification of the formal parameters ends with ellipses. That is, the general syntax is:

```
type_function  
    name_function (specification_explicit_parameters, ...);
```

Here is the specification of explicit parameters, number and types of which are fixed and known at compile time. These are called compulsory. After the list of explicit (compulsory) parameters optional comma is placed and then ellipses are placed in order to notify the compiler that further control of the conformity of the number and types of parameters in the processing of the function call should not be carried out.

Each function with the variable parameter list must have a mechanism of the determination of their number and types. There are two approaches to solving this task.

1. The first approach involves the addition at the end of the list of optional parameters a specific parameter-indicator with

unique value, which will signal the end of the list. In the body of the function parameters are sorted out consistently, and their values are compared with a previously known end sign.

2. The second approach involves the transmission of the value of the real number of actual parameters to the function. This value can be passed to the function by one of the explicitly specified (compulsory) parameters.

It should be noted that in both approaches — and when setting the end sign, and when specifying the number of actually used parameters — the transition from one actual parameter to another is performed using pointers, that is, using address arithmetic. Let us illustrate this by examples.

Example 1. *Compare the program to calculate the sum of the set of integers using a number of summands.*

```
#include <iostream>
using namespace std;

//Function prototype
long summa (int,...);
void main()
{
    cout << "\n summa(2,4,6)=" << summa (2,4,6);

    cout << "\n summa(6,1,2,3,4,5,6)="
        << summa (6,1,2,3,4,5,6);
}
//Transmit a number of parameters
long summa (int k,...)
{
    //pk contains the address of the location
```

```

//of the beginning of the parameters list.
//Due to the fact that the parameters
//are arranged in order
//in the memory
int *pk=&k;

//sum calculation
//k - number of parameters
long sm=0;
for (;k;k--)
    sm+=*(++pk);

return sm;
}

```

Comments

Normally, when describing the function a list of variables is given — a set of formal parameters, using which access to the passed values is provided. There is no such list. How can we get an access to the list of the passed parameters? This is done by using pointers: first an address of the end or the beginning of the list of explicit parameters is placed in the pointer, and then using this value, we produce moving on the variable parameter list.

Example 2. Rewrite the program using the predefined value (let it be 0).

```

#include <iostream>
using namespace std;
//k - this is now only
//one of summable parameters
long summa (int k,...)

```

```

{
    //beginning with k
    int *pk=&k;
    long sm=0;
    //we move
    //until not meet the value 0
    while (*pk)
        //sum calculation
        sm+=*(pk++);
    return sm;
}

void main()
{
    //test
    cout << "\n summa(4,6,0)="<< summa (4,6,0);
    cout << "\n summa(1,2,3,4,5,6,0)="
        << summa (1,2,3,4,5,6,0);
    cout << "\n summa(1,2,0,4,5,6,0)="
        << summa (1,2,0,4,5,6,0);
}

Result of the program work

summa(4,6,0)=10
summa(1,2,3,4,5,6,0)=21

//Attention here the function
//will finalize to first zero
//the remaining data will be lost
summa(1,2,0,4,5,6,0)=3

```

Another way of analyzing arguments within the function. This method will allow to analyze not only arguments of the arbitrary number, but with different types. At the beginning we connect the library `stdarg.h`, namely it contains the necessary automatic analysis tools. Let us examine the following actions step by step.

1. Declare a variable of type `va_list` (redetermined `char *`). This is where we will store a pointer to the list of parameters of our function.
2. Equability we call the function:

```
void va_start( va_list arg_ptr, prev_param );
```

The first parameter is a variable of the type `va_list` that was created beforehand, a pointer to the parameter list created by this function will be written in this variable. The second parameter of our function is necessary when `va_list` pushes off from something when we link `arg_ptr` with the list.

3. We analyze parameters using a multifold call of function:

```
type va_arg(va_list arg_ptr, type);
```

This function gets a parameter from the list, a pointer to which is transmitted to it as the first parameter, and rearranges this pointer to the next element. As a second parameter that you should take out of the list.

4. We are finishing the analysis of parameters and we clean the internal pointer, setting it in zero using the function:

```
void va_end(va_list arg_ptr);
```

Consider the example:

```
#include <iostream>
using namespace std;
#include <stdarg.h>
//function calculates the sum of numbers by the number of count
```

```

//typeof determines what type of elements will be used
//true - parameters of integral type (int)
//false - parameters of integral type (double)
void AnyType(int count,bool typeof,...){
    //sum of the integer parameters
    int sumi=0;
    //sum of the real parameters
    double sumd=0.0;
    //create a pointer to the list of parameters
    va_list arg_ptr;
    //we get this pointer,
    //pushing off from the first actual parameter
    va_start(arg_ptr,count);
    //miss the second actual parameter
    va_arg(arg_ptr,bool);
    //count times we read the following parameters
    while(count--){
        //if typeof is true, we read parameters
        //of the type int
        //if typeof is false, we read parameters
        //of the type double
        (typeof)?sumi+=va_arg(arg_ptr,int):
                sumd+=va_arg(arg_ptr,double);
    }
    //close the list of parameters
    va_end(arg_ptr);

    //if typeof is true, we show the sum of the type int
    //if typeof is false, we show the sum
    //of the type double
    (typeof)?cout<<"Integer sum =
                "<<sumi:cout<<"Double sum = "<<sumd;
    cout<<"\n\n";
}

void main()
{

```



```
//call the function to sum parameters
//of the type int
AnyType(4,true,3,8,9,4);
//call the function to sum parameters
//of the type double
AnyType(3,false,2.5,1.1,3.6);
}
```

As you have already noticed, the functions with an unlimited number of parameters have several disadvantages. In particular, the analysis of the parameters is labored and often even has a great code. In addition, the probability of an error at runtime for such functions is increased many times over, because it is very difficult to keep track of the correct parameter transfer. Therefore, we recommend you to carry out the implementation of this apparatus extremely carefully.

Homework

1. Write a class that implements the work with complex numbers.
2. Write a program that implements a class-based card game Black Jack.
3. Write a program that implements the function with an unlimited number of parameters that organizes formatted output to the screen.

